Distributed Systems & Service Architectures

Mastering Software Architecture Patterns and System Design

Rahul Rajat Singh

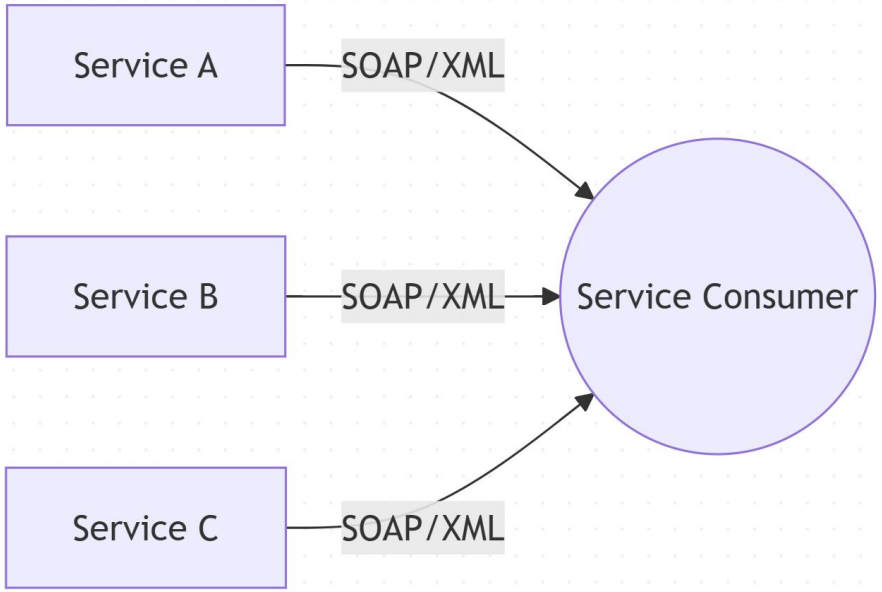
Service-Oriented Architecture (SOA)

Distributed Systems & Service Architectures

Mastering Software Architecture Patterns and System Design

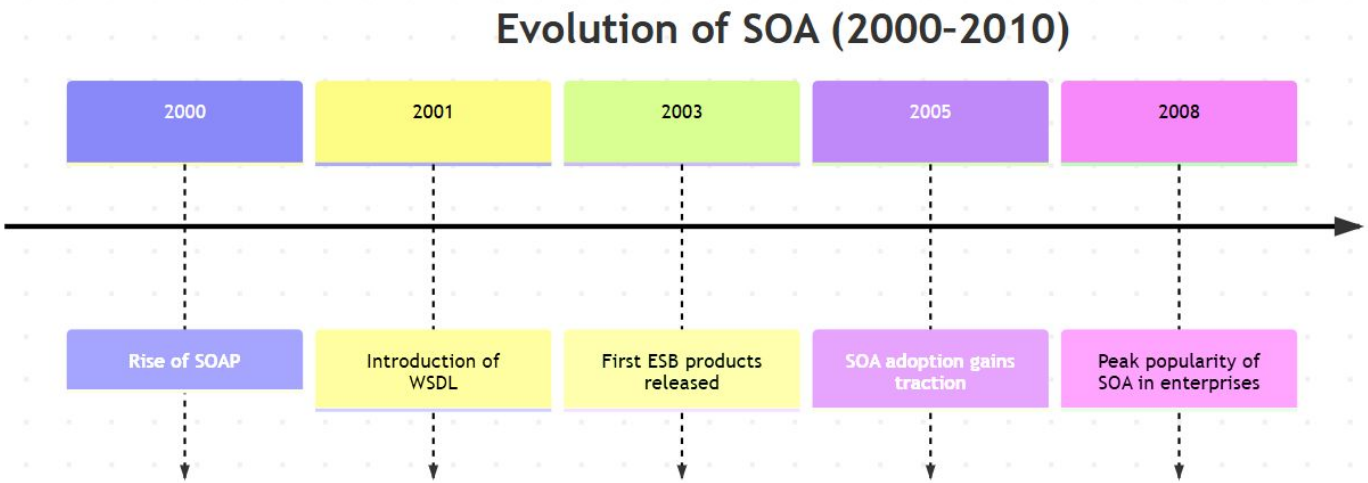
What is SOA?

- SOA is an architectural style that supports service reusability, loose coupling, and interoperability.
- Services expose well-defined interfaces, typically over a network.
- Services communicate using standardized protocols and formats.



Historical Context

- Emerged in early 2000s to solve integration challenges in enterprise systems.
- Grew out of need for B2B communication and reuse across siloed systems.
- Key players: IBM, Oracle, Microsoft.
- Closely tied to rise of XML and SOAP standards.

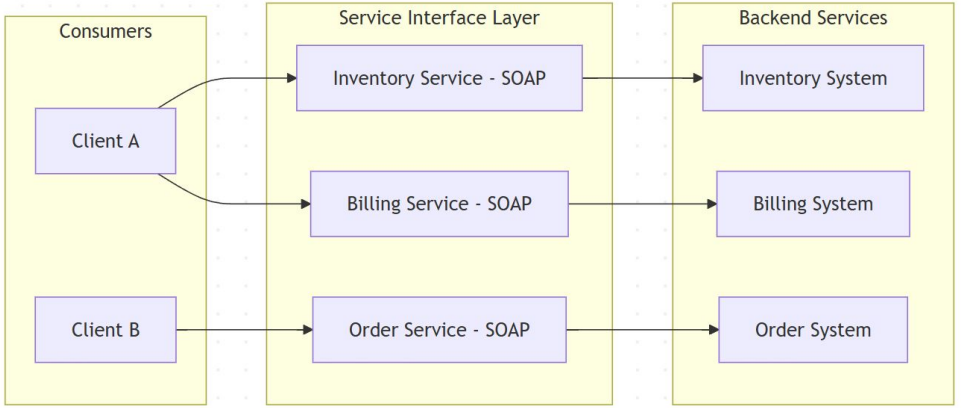


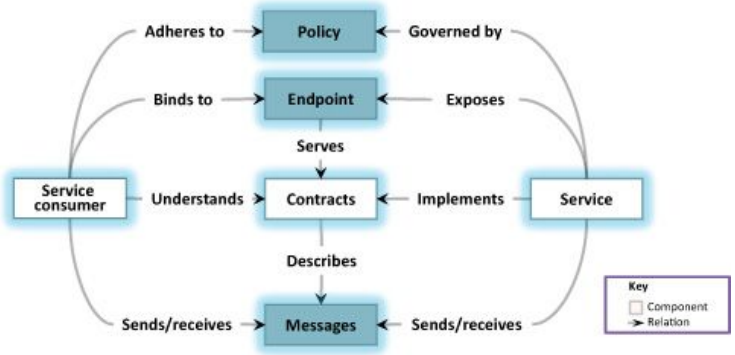
Core Principles of SOA

- Loose Coupling – Services operate independently.
- Interoperability – Works across platforms.
- Reusability – Services can be reused across apps.
- Discoverability – Services registered and searchable.
- Abstraction – Service hides implementation details.

High-Level SOA Architecture

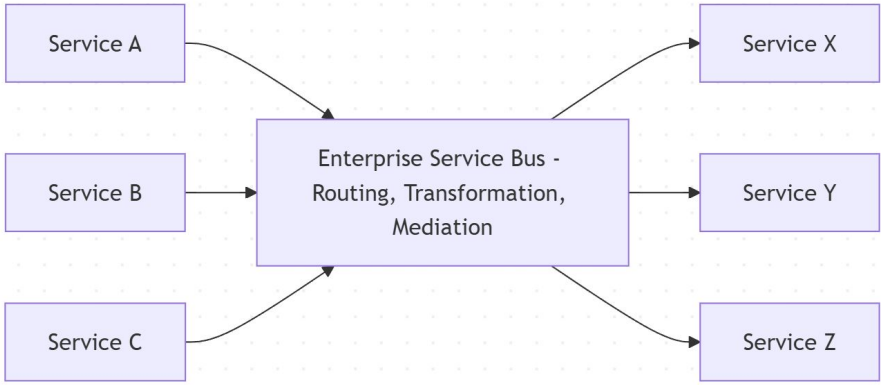
- Service consumers call services through a service contract.
- Services are exposed via endpoints.
- Messaging occurs over standard protocols.





Role of the Enterprise Service Bus (ESB)

- Middleware that routes, transforms, and mediates service messages.
- Helps with orchestration, security, monitoring, logging.
- Enables centralized control.



Pros and Strengths of SOA

- Enterprise-wide integration across platforms.
- Service reuse and agility.
- Well-suited for large, regulated, or structured environments.
- Centralized governance and monitoring.



Limitations of SOA

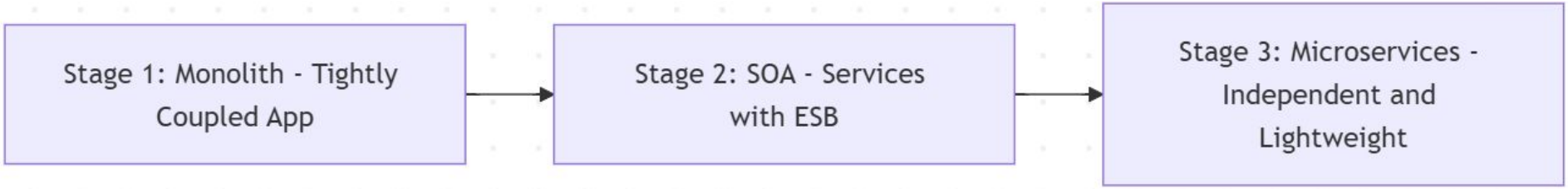
- Heavyweight protocols (XML/SOAP)
- Complexity of ESBs
- Centralized bottlenecks
- Governance overhead
- Slower development velocity

SOA vs. Microservices

- Communication Protocols
  - SOA: Uses heavyweight protocols like SOAP and data formats like XML
  - Microservices: Prefer lightweight protocols such as REST/JSON or gRPC
- Service Granularity
  - SOA: Typically coarse-grained services (e.g., BillingService, InventoryService)
  - Microservices: Fine-grained, single-responsibility services (e.g., InvoiceService, PaymentService)
- Integration Style
  - SOA: Often relies on a centralized ESB (Enterprise Service Bus) for routing, transformation, and mediation
  - Microservices: Communicate directly or via lightweight infrastructure (e.g., API Gateway, event brokers)
- Deployment & Scalability
  - SOA: Services are often co-deployed or share infrastructure, making independent scaling harder
  - Microservices: Each service is independently deployable, enabling horizontal scaling per service
- Governance & Control
  - SOA: Emphasizes centralized governance, often with strict contract and schema management
  - Microservices: Support decentralized governance, allowing teams to choose their own tech stacks and release cycles
- Technology Stack
  - SOA: Generally homogeneous, uses enterprise stacks (Java EE, .NET, WebSphere, etc.)
  - Microservices: Promotes polyglot programming, using multiple languages and frameworks across services
- Suitability
  - SOA: Ideal for large enterprises needing integration across legacy systems or B2B workflows
  - Microservices: Better suited for cloud-native, agile teams building modular, rapidly evolving systems

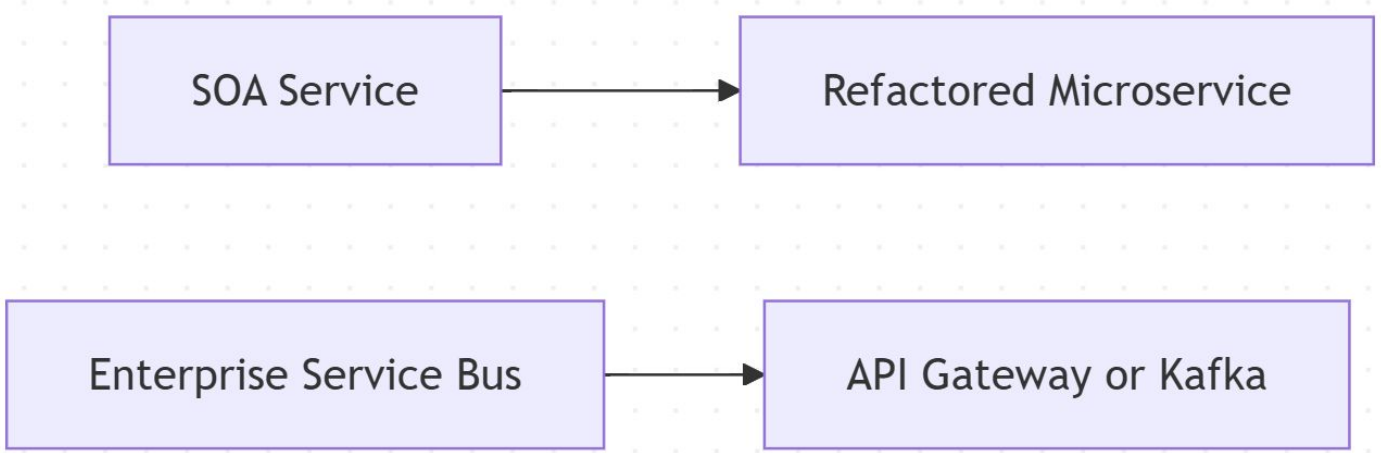
When is SOA Still Used Today?

- In large enterprises with legacy infrastructure.
- For B2B integrations with strict standards.
- In regulated industries needing SOAP (e.g., banking, healthcare).
- As the base of legacy monoliths being slowly modernized.



Modernization Path: SOA to Microservices

- Identify domain boundaries.
- Decompose coarse-grained services.
- Replace ESB with API Gateway / Service Mesh.
- Introduce REST/JSON or gRPC gradually.



Summary - Service-Oriented Architecture (SOA)

- SOA enabled enterprise integration at scale.
- Based on heavy but standardized protocols (XML/SOAP).
- Used centralized ESBs for control and mediation.
- Contrast: Microservices are more lightweight and decentralized.
- Many systems still run on SOA today, often evolving slowly.
- What’s next:
  - Microservices Architecture Overview

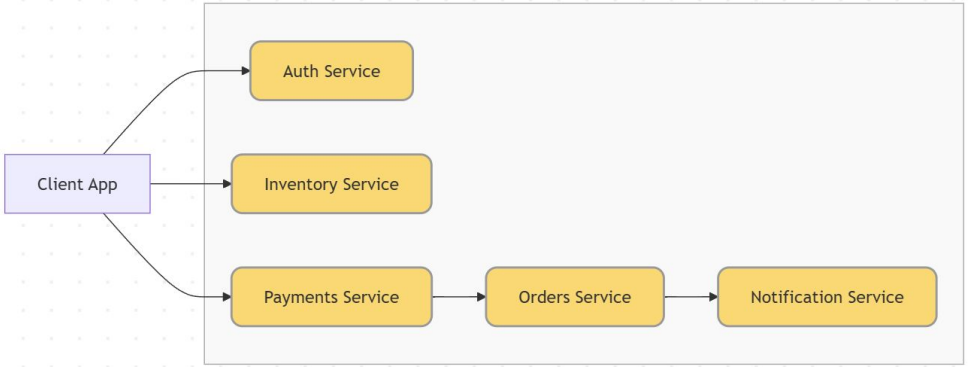
Microservices Architecture Overview

Distributed Systems & Service Architectures

Mastering Software Architecture Patterns and System Design

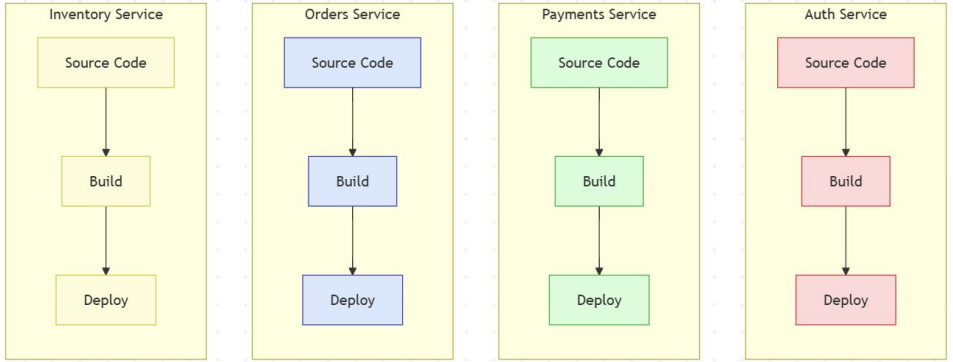
What are Microservices?

- Microservices is an architectural style where applications are composed of small, independent services.
- Each service focuses on a single business capability.
- Services communicate over lightweight protocols (e.g., REST, gRPC, messaging).
- Services are independently deployable and scalable.



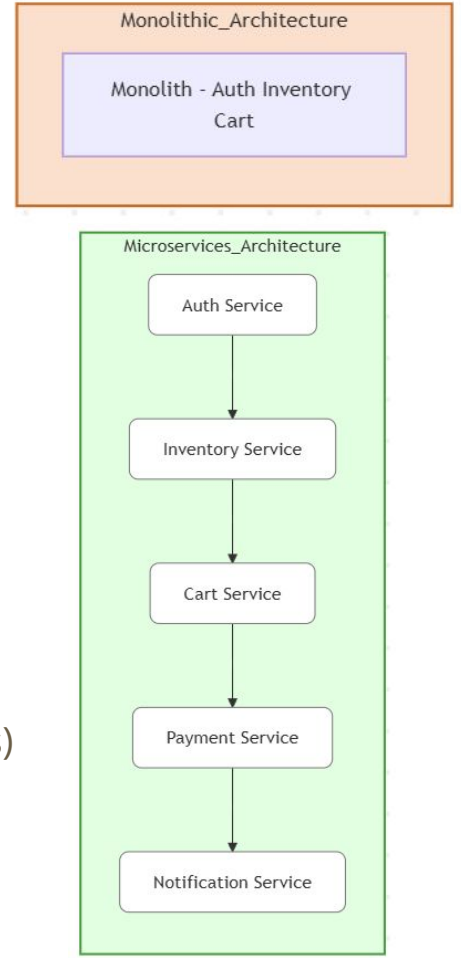
Key Characteristics of Microservices

- Single Responsibility: Each service owns one bounded context.
- Autonomous Deployment: Services can be deployed and updated independently.
- Language and Tech Stack Flexibility: Each team can choose their own tools (polyglot).
- Fault Isolation: Failure in one service doesn’t bring down the entire system.
- DevOps Friendly: Enables CI/CD and agile delivery.



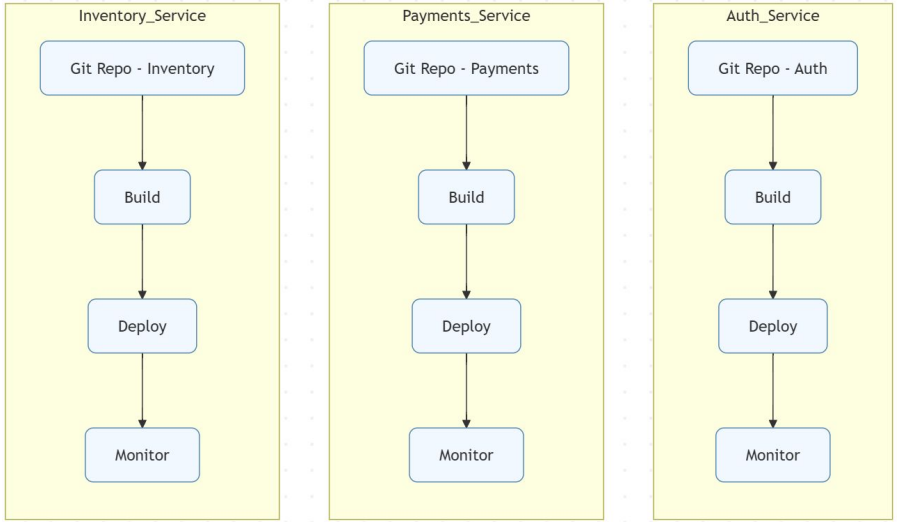
Microservices vs. Monolith

- Codebase
  - Monolith: Single, unified codebase
  - Microservices: Multiple, smaller codebases per service
- Deployment
  - Monolith: All-or-nothing deployment
  - Microservices: Independently deployable units
- Scalability
  - Monolith: Scales as a whole
  - Microservices: Scales per service as needed
- Technology Stack
  - Monolith: Single tech stack across the system
  - Microservices: Polyglot (different languages, DBs, frameworks)
- Failure Impact
  - Monolith: One failure can impact the entire app
  - Microservices: Fault isolation limits blast radius



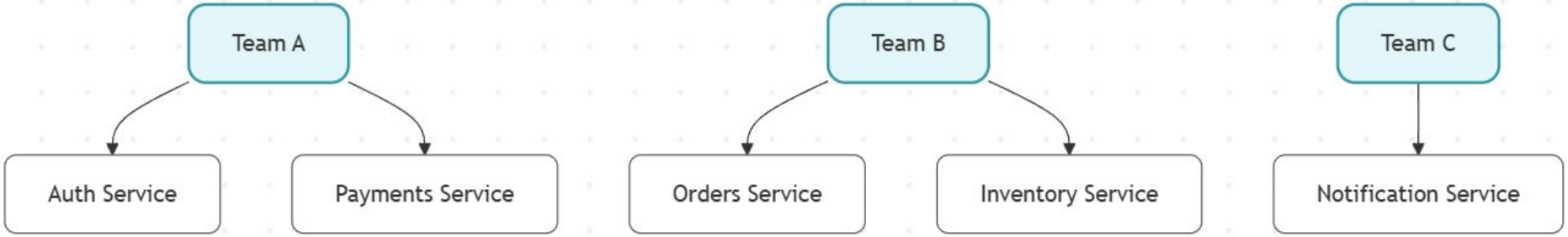
Deployment Autonomy

- Teams can build, test, and deploy services independently.
- Enables faster releases and hotfixes.
- Easier to implement CI/CD pipelines per service.
- Supports rolling, blue-green, or canary deployments.



Organizational Alignment

- Teams own services end-to-end: “You build it, you run it.”
- Maps well to Conway’s Law: Team boundaries = service boundaries.
- Encourages full-stack ownership and cross-functional collaboration.
- Promotes scalability in both system and team structure.

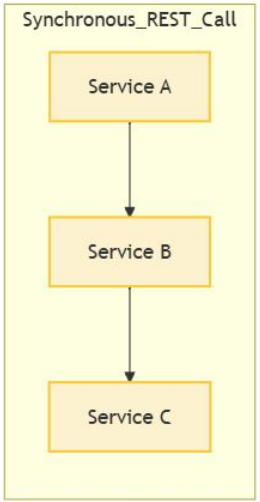


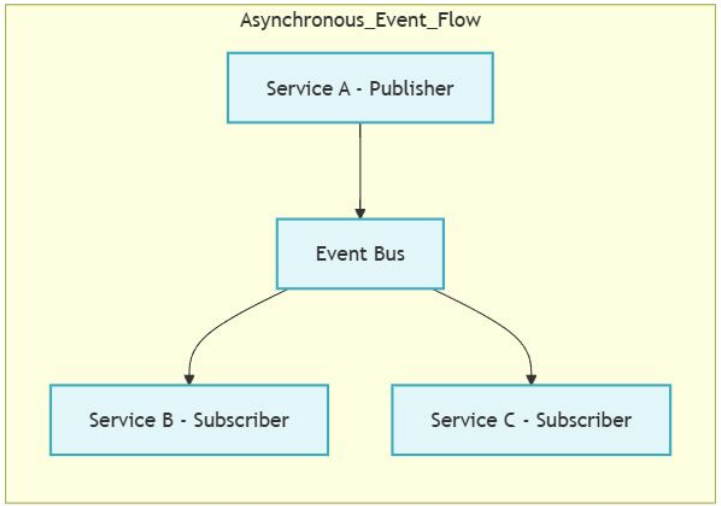
Technology Diversity & Polyglot Architecture

- Each service can choose its own:
  - Programming language
  - Database (SQL, NoSQL, etc.)
  - Frameworks & libraries
- Reduces tech lock-in and encourages best-fit tools per domain.
- Requires clear contracts and API specs to maintain interoperability.

Service Communication Patterns

- Synchronous: REST, gRPC
- Asynchronous: Messaging (Kafka, RabbitMQ, SQS)
- Services interact via APIs, contracts, and events.
- Must handle network failures and timeouts gracefully.





Observability in Microservices

- Logs, metrics, and traces must be aggregated and correlated.
- Distributed tracing is essential (e.g., OpenTelemetry, Jaeger).
- Monitoring each service’s health, performance, and dependencies.

Common Challenges

- Increased system complexity
- Service discovery and communication overhead
- Data consistency and distributed transactions
- Monitoring and debugging across services
- Organizational silos if not coordinated

When to Use Microservices

- You need frequent deployments by multiple teams
- System is growing in scale and complexity
- You need independent scaling of different parts
- Teams want ownership and autonomy
- You’re building a cloud-native, modular system

Summary

- Microservices = small, focused, independently deployable services
- Enables agility, scale, and team autonomy
- Requires investment in DevOps, observability, and good API design
- Best for large, evolving systems — not always the right choice for small teams or early-stage products
- What's next:
  - Modular Monoliths and Migration Strategy

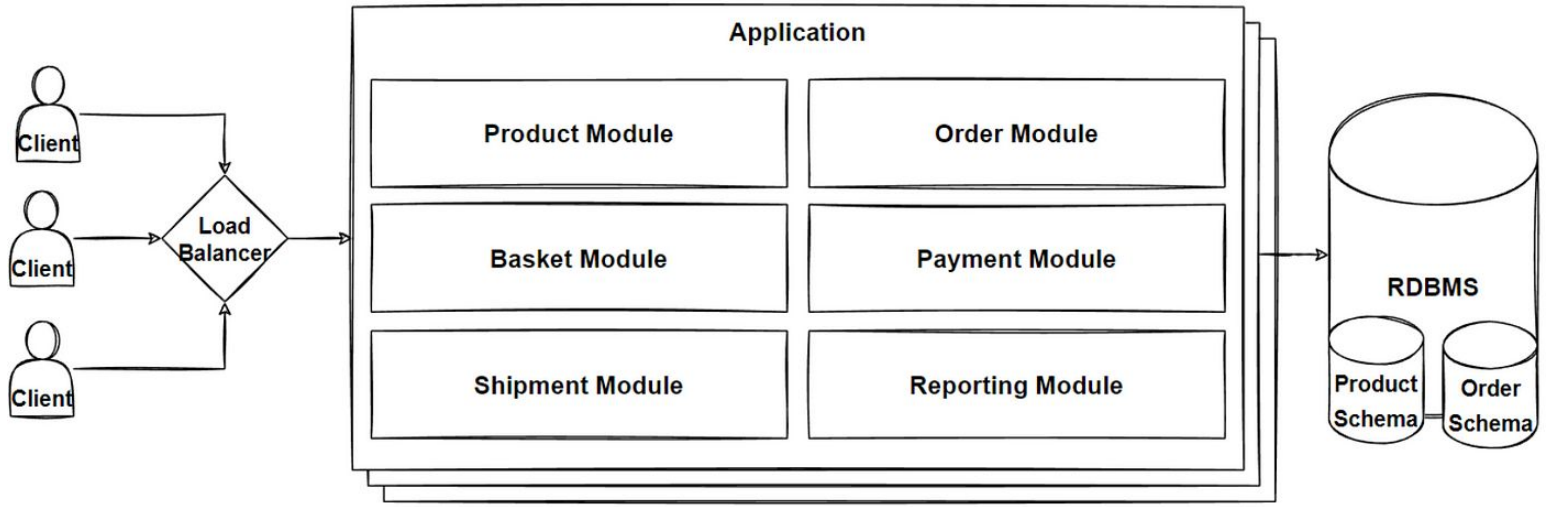
Modular Monoliths and Migration Strategy

Distributed Systems & Service Architectures

Mastering Software Architecture Patterns and System Design

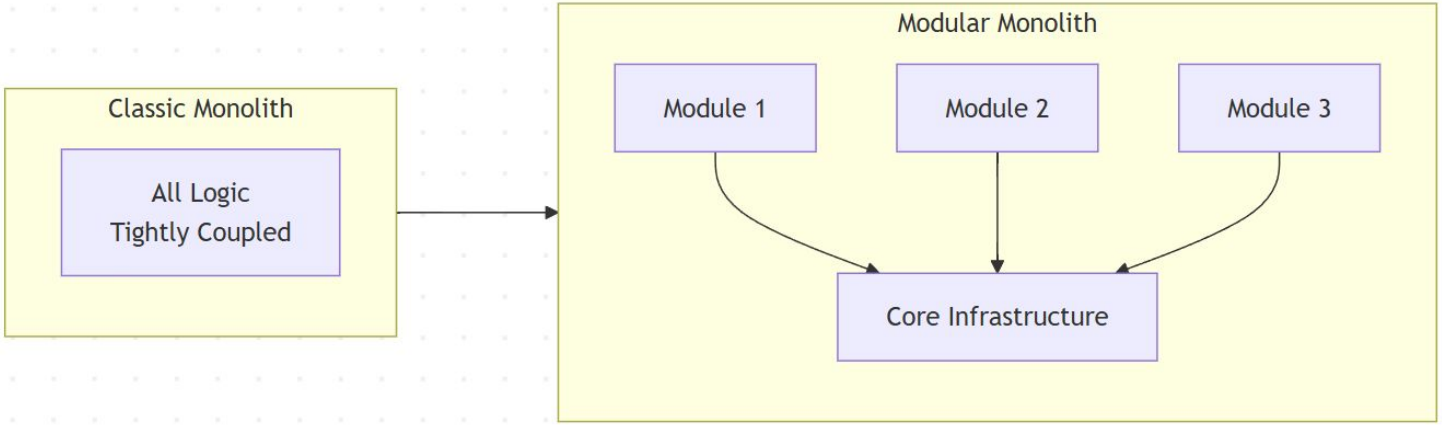
What is a Modular Monolith?

- A monolith where internal modules are clearly separated by well-defined boundaries
- Each module encapsulates business logic + data access
- All modules run in the same process (single deployable unit)



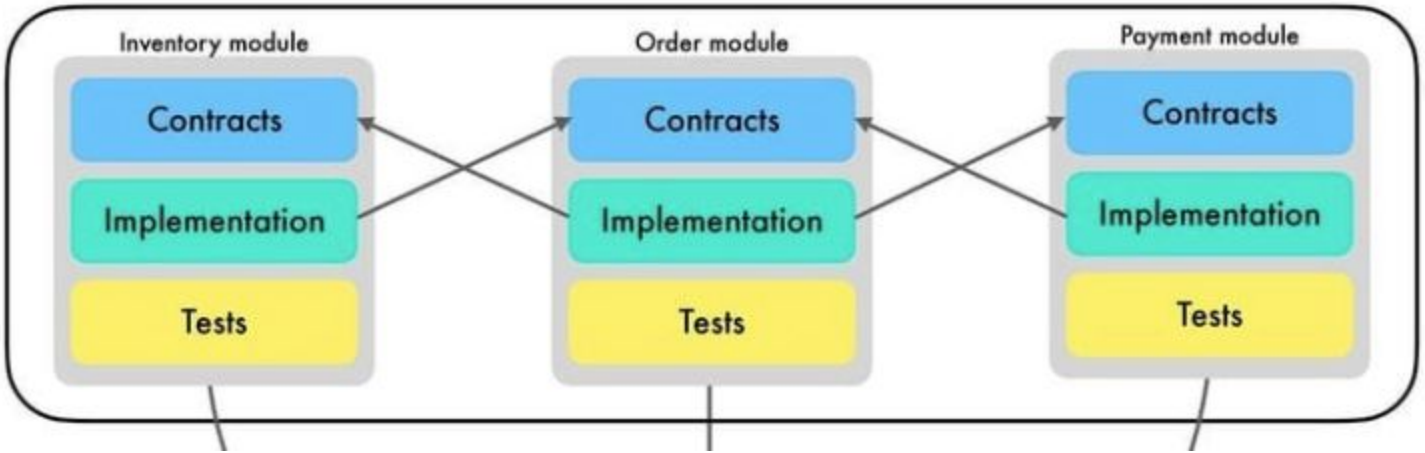
Key Characteristics

- Single deployable unit
- Enforces module boundaries in codebase (e.g., namespaces, packages, folders)
- Modules interact via method calls or internal APIs
- Enables unit testing and reuse within modules



Why Modular Monoliths?

- Don’t pay the distributed systems tax early
- Encourages discipline around boundaries
- Fast local development & debugging
- Works well with small teams and tight feedback loops



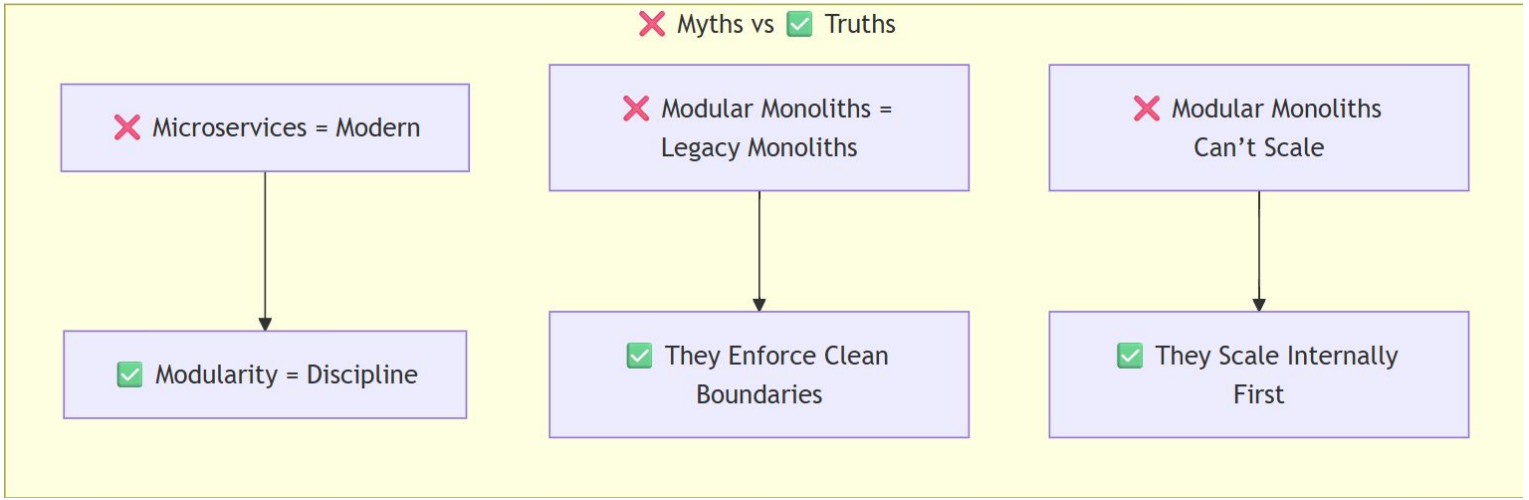
Ideal Use Case – Small Teams

- Small team (3–10 devs)
- Need to move fast without infra overhead
- Domain is complex enough to modularize, but not yet distributed



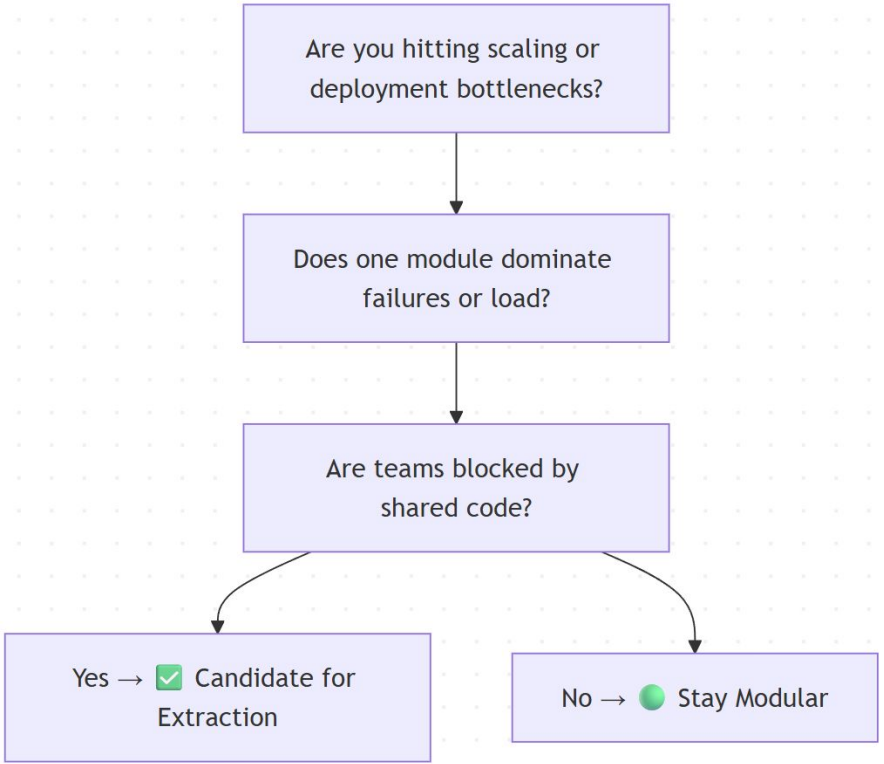
Common Misunderstandings

- “Modular Monolith = Bad Monolith” ❌
- “Modular Monolith is just a microservice in disguise” ❌
- “It can’t scale” ❌ — it can scale modularly before it needs to scale distributedly



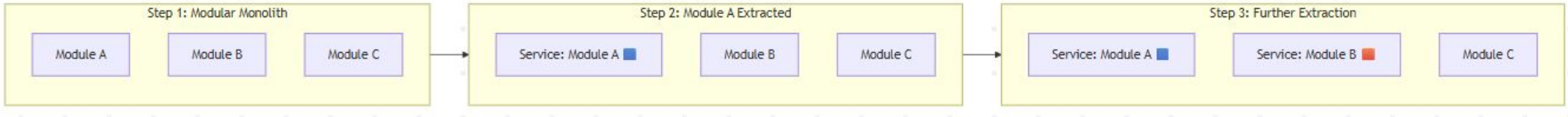
Migration Triggers

- When does it make sense to break out to microservices?
  - Module has independent scaling needs
  - Deployment is blocked by unrelated changes
  - Module needs different tech stack
  - Clear team ownership boundaries form



Migration Paths

- Strangler Fig Pattern: Extract one module at a time, keeping the rest in monolith
- Use internal API contracts to prepare modules for isolation
- Replace internal calls with HTTP/gRPC as needed

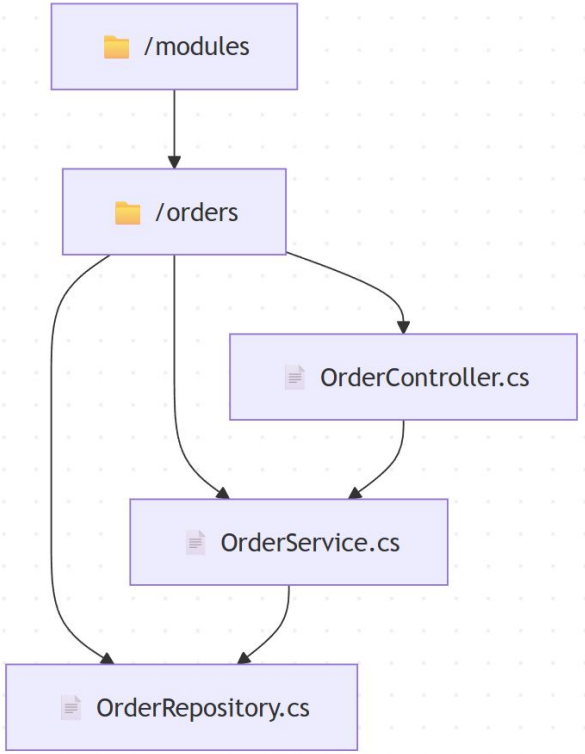


Best Practices

- Enforce modularity in code (e.g., package boundaries, folder rules)
- Use domain-driven design (DDD) principles
- Write adapters/ports even inside monolith
- Don’t rush — maturity before microservices

Code Organization Tips

- Structure modules around business domains, not technical layers
- Each module = its own:
  - Application layer
  - Domain logic
  - Persistence abstraction



From Modular to Microservices – Checklist

- ✅ Independent domain logic
- ✅ Clear API surface
- ✅ No shared DB
- ✅ Separate deploy pipeline
- ✅ Separate monitoring/logging

Summary – Modular Monoliths & Migration

- Modular monolith = structure + speed + clarity
- Avoids premature complexity
- Ideal for teams that want scale by discipline
- Migration is a choice, not a necessity
- What’s next:
  - Micro Frontends & UI Composition

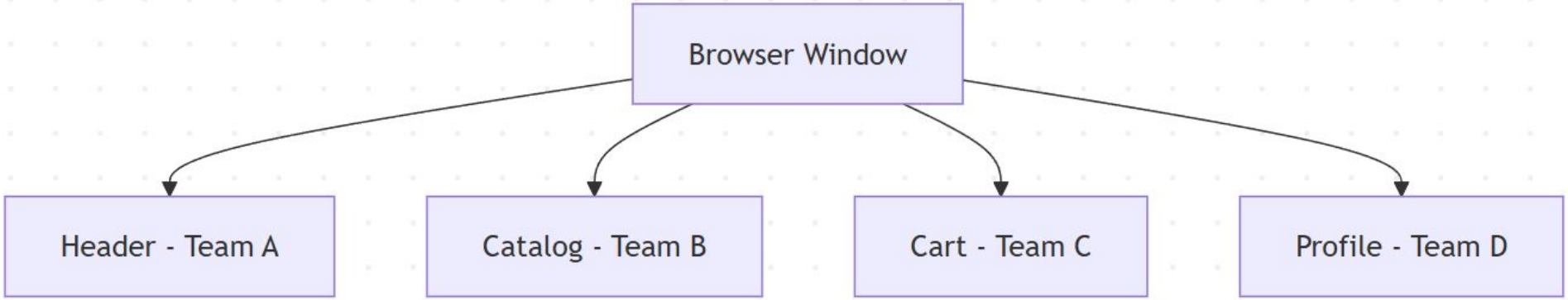
Micro Frontends & UI Composition

Distributed Systems & Service Architectures

Mastering Software Architecture Patterns and System Design

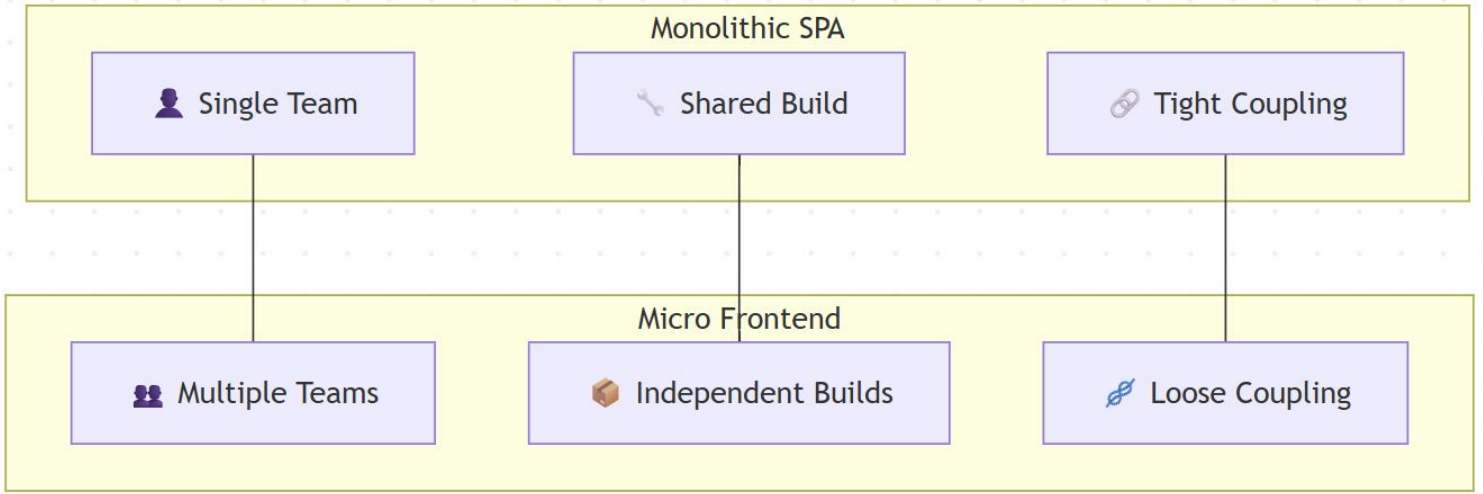
What Are Micro Frontends?

- Applying microservice principles to frontend code
- Teams own end-to-end features (frontend + backend)
- Multiple frontend applications working as one



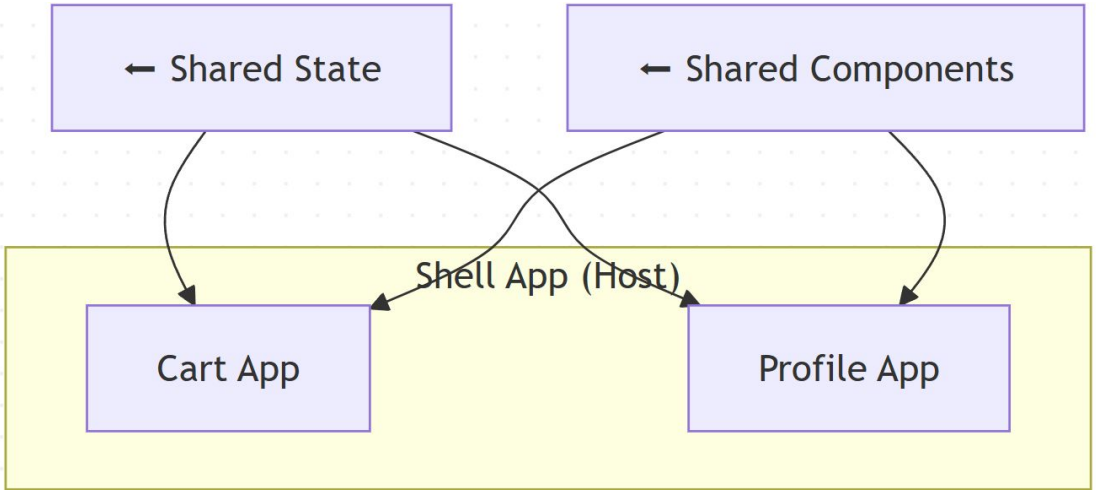
Why Micro Frontends?

- Scale frontend development across teams
- Allow autonomous deployment of features
- Reduce coordination overhead between teams
- Improve maintainability in large SPAs



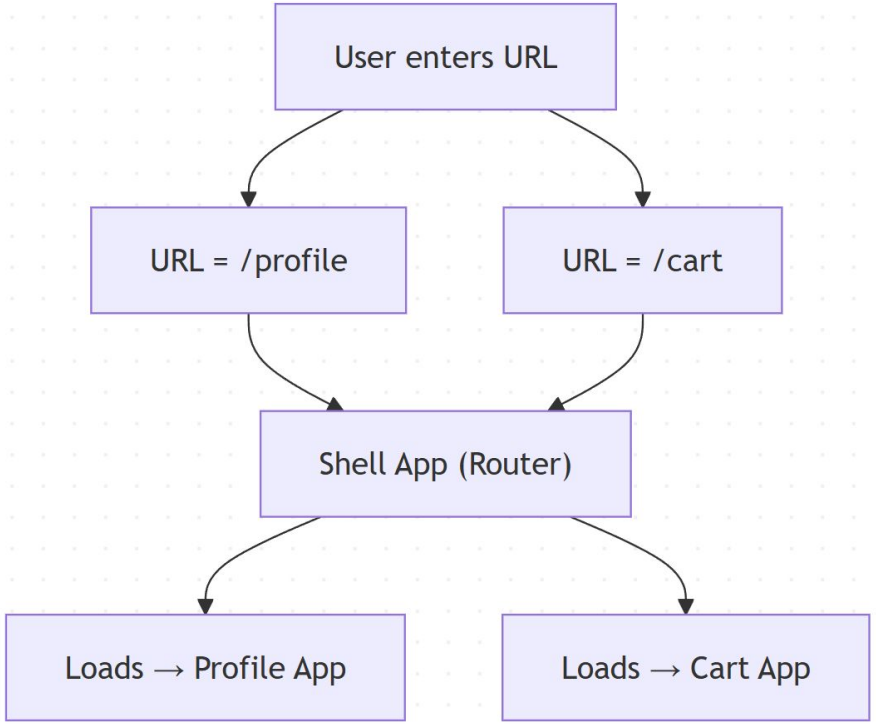
Typical Micro Frontend Architecture

- Shell app (host) + independently deployed remote apps
- Shared components (e.g., design system, auth)
- Communication through events or shared state



Frontend Routing in Micro Frontends

- Shell-based routing (host handles all routes)
- Independent routing (each MFE handles its own sub-route)
- Fragment-based (composition of sub-routes)



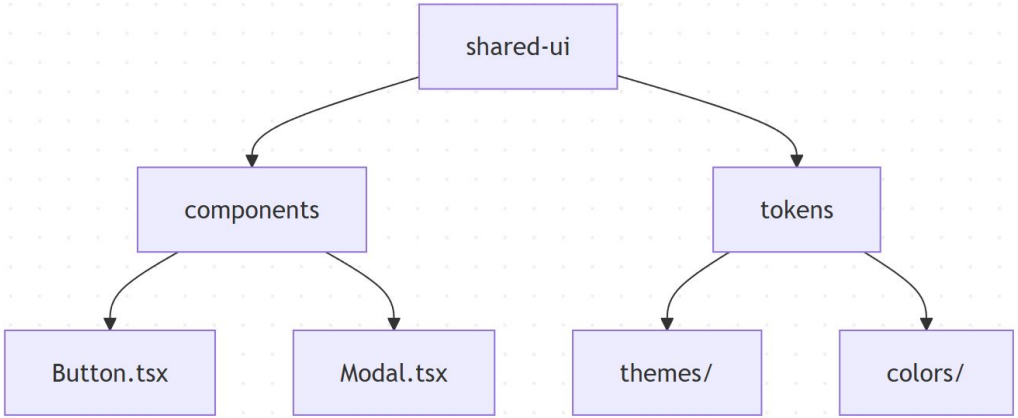
Communication Between Micro Frontends

- URL-based (deep linking, query params)
- Event bus (custom DOM events, RxJS, Redux, etc.)
- Shared state manager (e.g., Zustand, Recoil, Redux)
- Backend-driven sync (e.g., user info, feature flags)



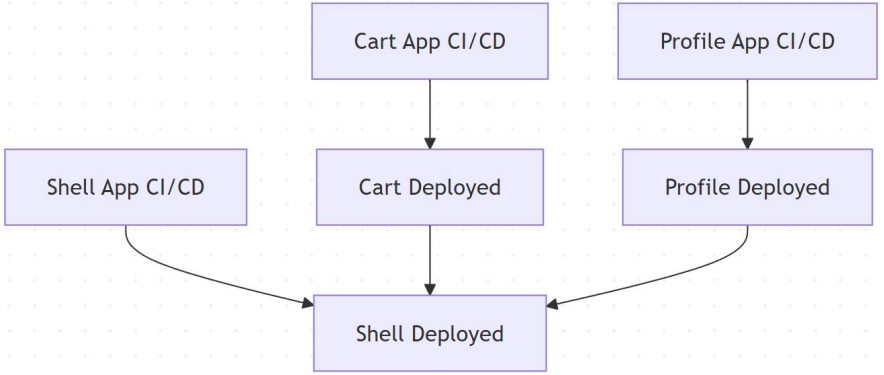
Shared Component Strategy

- Shared design system (e.g., buttons, inputs)
- Versioned components or libraries (via NPM)
- Avoid tight coupling via shared runtime state



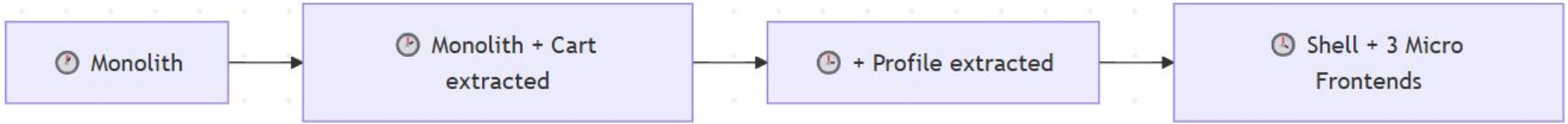
Deployment Models

- Single repo vs multi-repo (monorepo vs polyrepo)
- Independent CI/CD pipelines per micro app
- Deployment via:
  - Webpack Module Federation
  - iFrames
  - Edge-side includes (ESI)
  - CDN + Shell loader



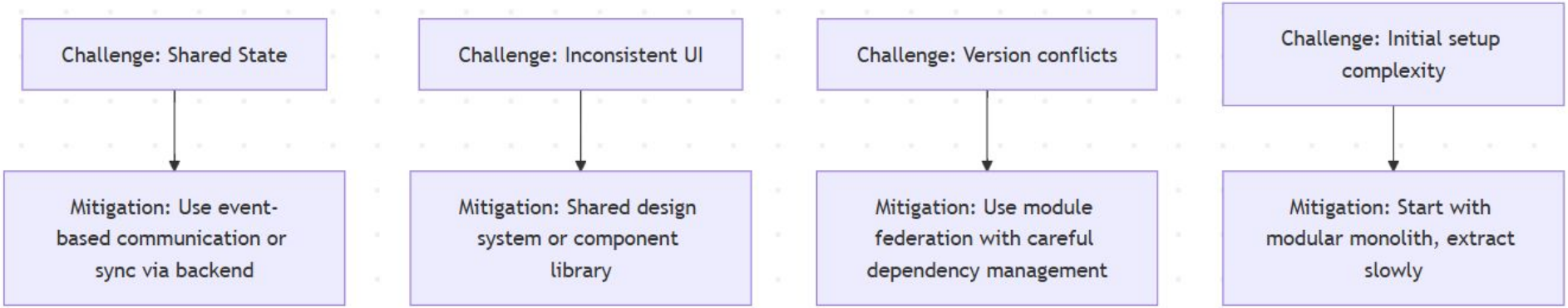
Example Evolution Path

- Phase 1: Monolithic frontend (e.g., React SPA)
- Phase 2: Extracted Profile & Cart as separate apps
- Phase 3: Shell loads them dynamically
- Phase 4: Team-based ownership and deployment



Challenges in Micro Frontends

- Initial setup complexity
- Consistency in UX and state
- Shared dependency/version management
- Performance overhead



Best Practices

- Favor composition over duplication
- Minimize shared runtime state
- Enforce UI consistency via design systems
- Avoid tight coupling between micro apps
- Start modular, then extract

Summary – Micro Frontends & UI Composition

- Micro Frontends = frontend modularization strategy
- Enables scale, autonomy, and independent deployments
- Shell app + micro apps + communication layer
- Start with modular monolith → extract when needed
- What's next:
  - Real-World Use Cases for Distributed Styles

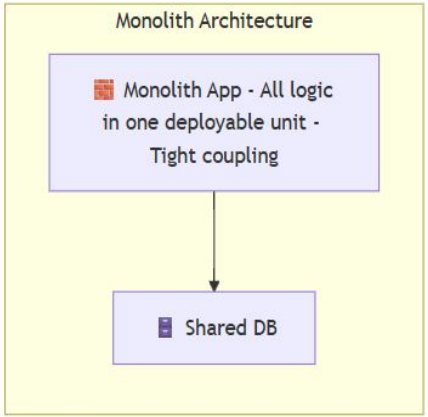
Real-World Use Cases for Distributed Styles

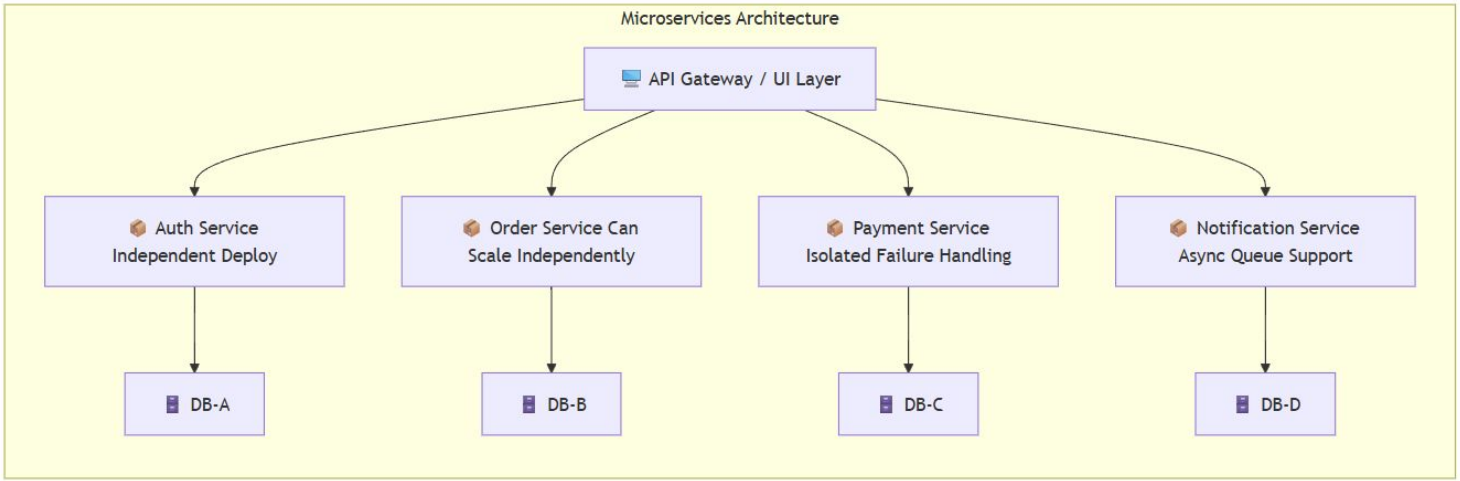
Distributed Systems & Service Architectures

Mastering Software Architecture Patterns and System Design

Why Use Distributed Styles?

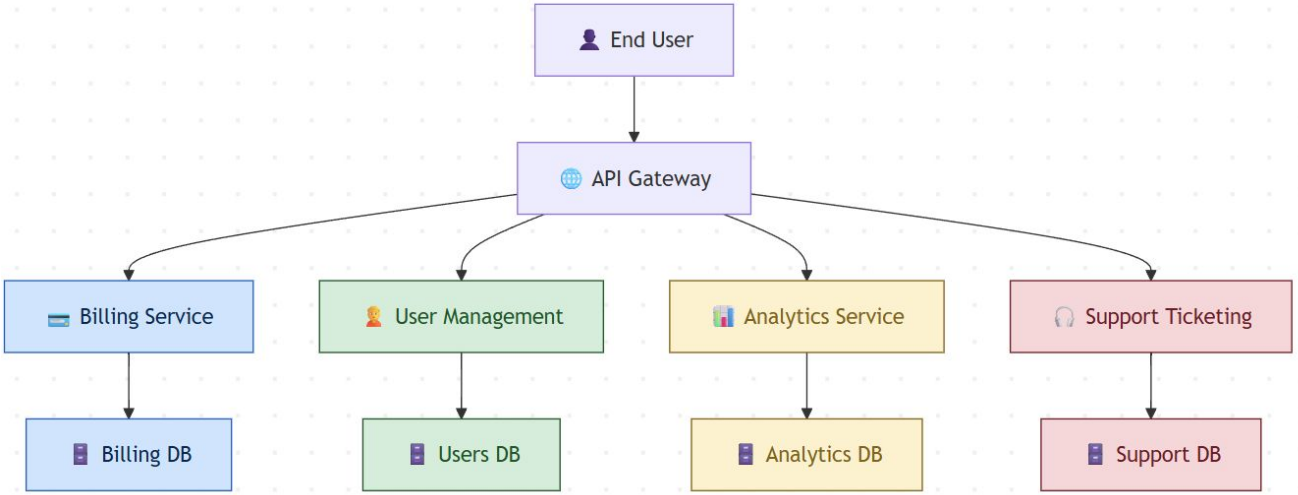
- Teams want independence
- Systems need scalability, fault isolation, deployment flexibility
- Monoliths hit limitations beyond a point
- Distributed styles enable specialization, reuse, and resilience





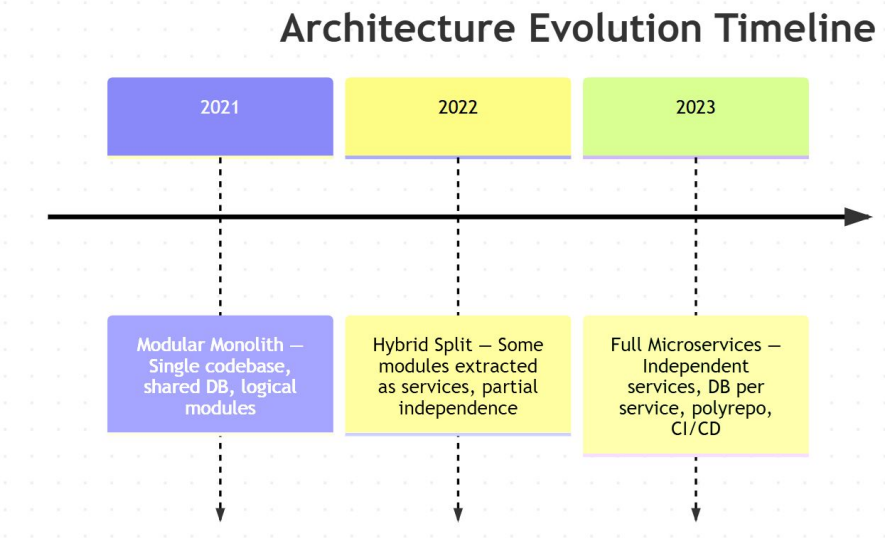
Multi-Team SaaS Platforms

- SaaS platforms often serve multiple customer segments, geographies, or modules (billing, users, analytics, support)
- Each module managed by an autonomous team
- Ideal for Modular Monolith → Microservice evolution



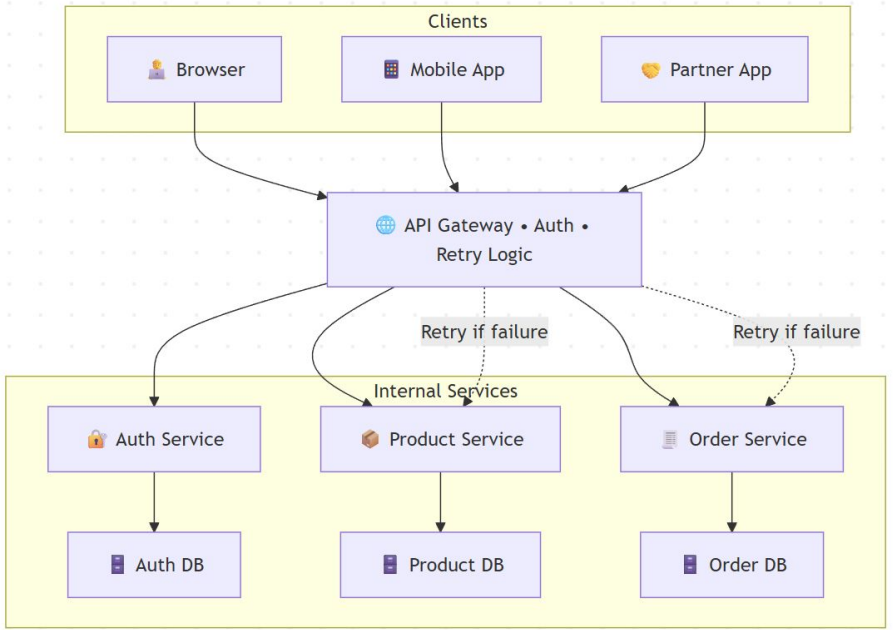
Visualizing Module-to-Service Evolution

- Teams start with shared codebase → gradually split out services
- Database per service becomes reality over time



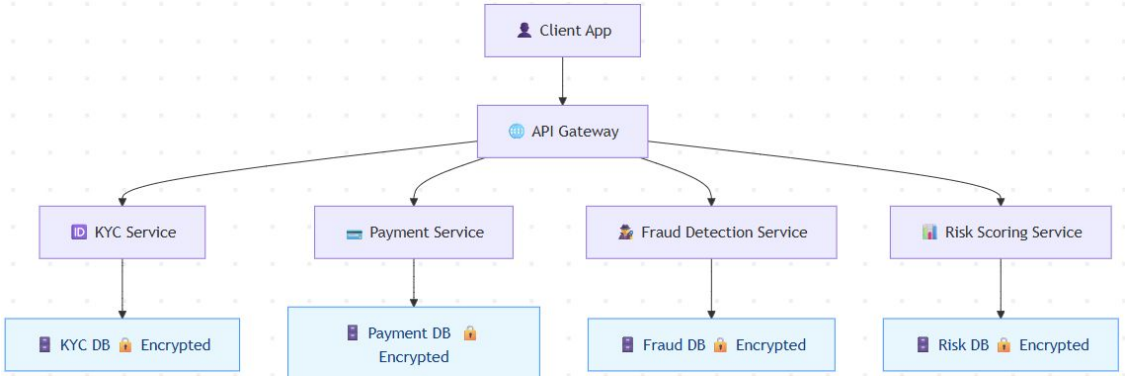
Scalable Public APIs – Platform Thinking

- Platforms like Stripe, Twilio, Shopify offer APIs for public consumption
- High availability, rate limiting, backward compatibility are critical
- API Gateways and service mesh often used



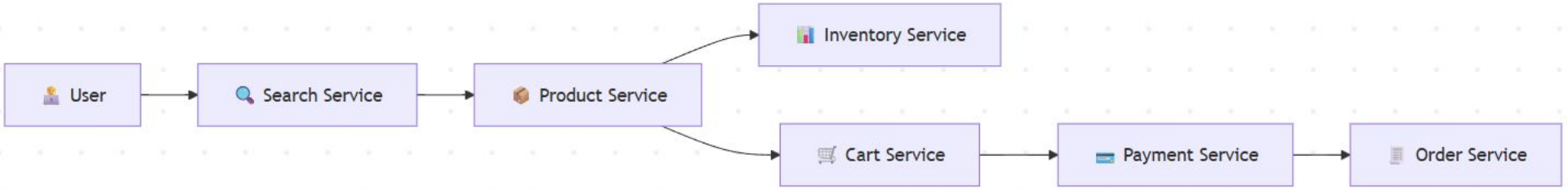
Microservices in Fintech – Real Demands

- Fintech platforms require:
  - Strong isolation (security & compliance)
  - Clear audit trails
  - Real-time + batch processing
- Services:
  - KYC Service
  - Payments
  - Fraud Detection
  - Risk Scoring



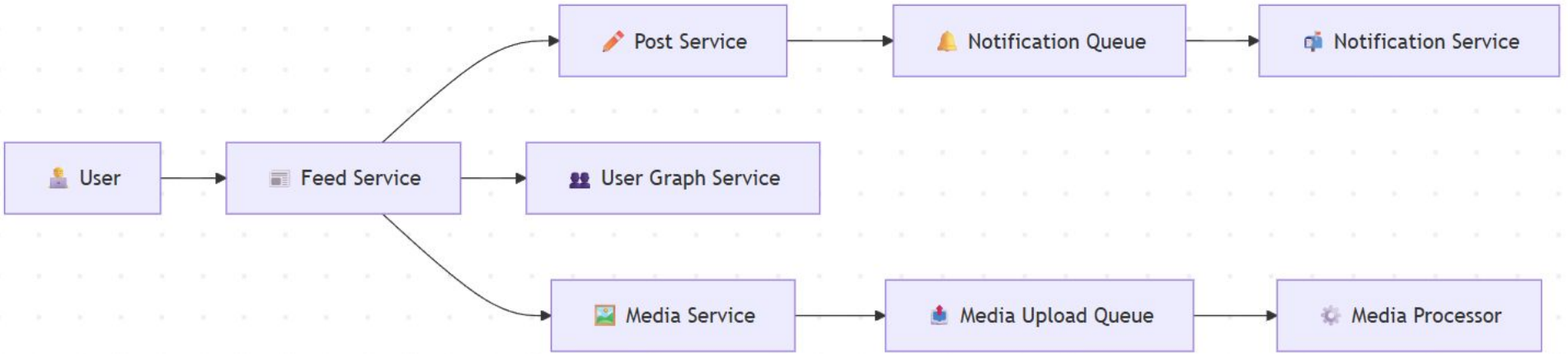
Microservices in E-Commerce

- E-Commerce platforms (Amazon, Flipkart):
  - Catalog Service
  - Inventory
  - Pricing Engine
  - Recommendation Engine
  - Order Fulfillment



Microservices in Social Platforms

- Social apps need:
  - Feed generation (high-volume read)
  - Messaging (low-latency, real-time)
  - Media processing (video, images)
  - Search and discovery



Challenges in Real-World Distributed Systems

- Network latency
- Eventual consistency
- Debugging across services
- Coordination and versioning
- Developer onboarding complexity

Summary - Real-World Use Cases for Distributed Styles

- Distributed styles solve scale, agility, and team independence
- Real-world systems balance trade-offs between monoliths and microservices
- Key is modular thinking, API discipline, and team ownership
- What's next:
  - Communication & Integration Patterns